3DG ENGINE

Audio & Particle Systems

Editor and Gameplay Scripting Guide

A practical guide to creating, configuring, previewing, and controlling sound and effects in 3DG Engine.

VERSION 1.0 • JULY 2026

SCOPE Covers the Audio Editor, Audio Mixer, Audio Source component, Particle Editor, Particle System component, no-code triggers, animation events, and the engine::Script helper API.

How to Use This Guide

Fast paths for artists, level designers, and gameplay programmers

The editor sections explain how to create and tune content without code. The scripting sections show how to drive that content from gameplay. Examples use the public engine::Script API so they remain safe when a component or runtime service is unavailable.

Choose a path

Goal	Start here	Then use
Create or edit a sound	Audio Editor	Audio Source + Audio Mixer
Place sound in a level	Audio Source component	Triggers, animation events, or scripts
Build a visual effect	Particle Editor	Particle System / Particle Effect component
Drive effects from gameplay	Scripting quick start	API reference + recipes
Fix a silent or invisible effect	Troubleshooting	Validation checklist

Recommended workflow

1. Create or import the audio clip or particle preset.
2. Add the matching component to a scene object and configure its default behavior.
3. Preview the component in the editor before entering Play mode.
4. Add a no-code trigger, animation event, or script only when gameplay must control it.
5. Test in Play mode with the Console, Audio Mixer, particle statistics, and bounds debug views visible.

IMPORTANT Do not call PlayAudio or BurstParticles every frame. Trigger them once from an input edge, collision/trigger event, animation event, timer, or state transition.

Naming conventions used in examples

- ExplosionAudio — an object with an Audio Source component.
- ExplosionFX — an object with a Particle System or Particle Effect component.
- Player — the scene object used for trigger-entry and trigger-exit checks.
- Runtime names are case-sensitive in practice; keep them unique and stable.

Part I — Audio

Create, mix, place, and control sound

Audio system at a glance

Tool or component	Purpose
Audio Editor	Generate simple clips, load audio, edit waveforms, preview, and export WAV files.
Audio Source	Attach a clip to an object and define playback, looping, spatialization, volume, pitch, bus, and attenuation.
Audio Mixer	Balance buses, mute groups, apply snapshots, duck music for dialogue, and tune bus processing.
Trigger Audio Action	Play, restart, pause, resume, or stop an Audio Source when a trigger is entered or exited.
Scripts / animation events	Synchronize playback with gameplay and animation.

Create or edit a clip

1. Open Panels > Audio Editor.
2. Generate a Sine Tone, Square Tone, or Noise clip; or load the selected audio asset from the Content browser.
3. Drag the selection range over the waveform to isolate the part you want to edit.
4. Apply Trim, Reverse, Gain, Normalize, Fade In, or Fade Out. Use Undo, Redo, or Reset Original while experimenting.
5. Use Preview Edited Clip to listen, then Export to Project. Edited output is exported as WAV.

Supported editor inputs

The Audio Source picker accepts WAV, FLAC, MP3, and OGG assets. Decoder availability can vary by build; WAV is the safest interchange and export format.

> **TIP** Normalize first, then apply fades and final gain. This produces predictable peak level and avoids amplifying a fade twice.

Audio Source Setup

Add and configure the component

1. Select the scene object that should emit sound.
2. Open the component Add menu and add Audio Source.
3. Choose or drag an audio clip into Audio Clip.
4. Set Volume, Pitch, Mixer Bus, Spatial (3D), Loop, and Autoplay.
5. For spatial audio, tune Min Distance, Max Distance, and Rolloff while moving the listener through the level.
6. Use Preview and Stop Preview in edit mode; use Runtime Transport in Play mode.

Key settings

Setting	Meaning	Useful starting point
Enabled	Allows the runtime audio system to use this source.	On
Volume	Source gain before bus and master volume.	1.0; editor range 0–2
Pitch	Playback pitch/rate multiplier.	1.0; editor range 0.25–4
Mixer Bus	Routing group for volume, mute, snapshots, and effects.	SFX for gameplay sounds
Spatial (3D)	Positions sound in world space relative to the listener.	On for world sounds; off for UI/music
Loop	Repeats the clip until stopped.	Off for one-shots
Autoplay	Starts when runtime playback initializes.	On for ambience; off for events
Min Distance	Distance before attenuation begins.	1.0
Max Distance	Outer useful hearing distance.	30.0
Rolloff	Strength of distance attenuation.	1.0

ONE-SHOT REPLAY Use PlayAudio(true) when the same source may be triggered again before it finishes. The restart flag rewinds it before playback.

Audio Mixer and Snapshots

Mixer buses

Route every Audio Source to the bus that matches its role. Each bus has independent volume and mute controls; Master affects the complete mix.

Bus	Typical content
Master	Final output level
Music	Score and musical layers
SFX	Weapons, impacts, movement, machinery
Dialogue	Speech, narration, radio
UI	Menus, confirmations, notifications
Ambient	Wind, room tone, environmental loops

Snapshots

Preset	Use
Default	Return the mix to normal gameplay.
Paused	Reduce or filter gameplay audio while the pause menu is open.
Underwater	Apply an underwater-style filtered mix.
Indoor	Shift the mix for enclosed spaces.
Cinematic	Prioritize authored presentation during cutscenes.

Bus processing

Music, SFX, Dialogue, UI, and Ambient buses expose low-pass and high-pass filters, reverb wet/decay, and compressor threshold/ratio. Use Reset Processing to return a bus to neutral values.

DIALOGUE Enable dialogue ducking when speech should temporarily lower Music. Use a snapshot for broader scene changes; use ducking for speech intelligibility.

Audio Scripting Quick Start

Attach a script

1. Create a class derived from engine::Script.
2. Register the script name with ScriptRegistry during game startup, before Play mode.
3. Attach the registered script to a scene object in the Inspector.
4. Place an Audio Source on that same object for self-targeting helpers, or use FindObject for another source.

Complete example — play sound and particles on trigger entry

```
#include <engine/gameplay/Script.h>
#include <engine/gameplay/ScriptRegistry.h>
#include <memory>

class ImpactFeedbackScript final : public engine::Script {
public:
    void OnCreate() override {
        SetAudioBus(engine::AudioBus::SFX);
        SetAudioVolume(0.9f);
        SetAudioSpatial(true);
        SetAudioAttenuation(1.0f, 25.0f, 1.0f);
    }

    void OnUpdate(float dt) override {
        (void)dt;
        const auto player = FindObject("Player");
        if (player != engine::ecs::kNull && WasTriggerEntered(player)) {
            PlayAudio(true);
            BurstParticles(64);
        }
    }
};

void RegisterImpactFeedbackScript() {
    engine::ScriptRegistry::Instance().Register(
        "ImpactFeedbackScript",
        [] { return std::make_unique<ImpactFeedbackScript>(); });
}
```

COMPONENTS Attach both Audio Source and Particle System to the scripted object. The self-targeting helpers operate on Self().

Audio Script API

Every source-control method has a self-targeting overload and an overload that accepts engine::ecs::Entity. Most control methods return bool so gameplay can handle a missing component or unavailable runtime safely.

Method	Purpose / behavior
PlayAudio(restart = false)	Start or continue playback. Pass true to rewind first.
PauseAudio() / ResumeAudio()	Pause without losing the cursor, or continue from it.
StopAudio()	Stop playback and return the source to its stopped state.
SeekAudio(seconds)	Move the playback cursor to a time in seconds.
IsAudioPlaying() / IsAudioPaused()	Query runtime transport state.
AudioCursorSeconds()	Read the current playback cursor.
SetAudioVolume(volume)	Set source volume; values are clamped to 0 or greater.
SetAudioPitch(pitch)	Set pitch/rate; values are clamped to at least 0.01.
SetAudioLooping(looping)	Enable or disable repeat playback.
SetAudioSpatial(spatial)	Switch between world-space and non-spatial playback.
SetAudioBus(bus)	Route to Master, Music, SFX, Dialogue, UI, or Ambient.
SetAudioAttenuation(min, max, rolloff)	Set spatial falloff; invalid ranges are corrected safely.
ApplyAudioSnapshot(preset, transition)	Blend to Default, Paused, Underwater, Indoor, or Cinematic.
SetDialogueDucking(enabled)	Enable or disable music ducking for dialogue.

Target another object

```
const auto source = FindObject("ExplosionAudio");
if (source != engine::ecs::kNull) {
    SetAudioBus(source, engine::AudioBus::SFX);
    SetAudioVolume(source, 1.0f);
    PlayAudio(source, true);
}
```

Audio Gameplay Recipes

Indoor snapshot zone

```
void OnUpdate(float dt) override {
    (void)dt;
    const auto player = FindObject("Player");
    if (player == engine::ecs::kNull) return;

    if (WasTriggerEntered(player)) {
        ApplyAudioSnapshot(engine::AudioSnapshotPreset::Indoor, 0.35f);
    }
    if (WasTriggerExited(player)) {
        ApplyAudioSnapshot(engine::AudioSnapshotPreset::Default, 0.35f);
    }
}
```

Pause and resume a loop

```
const auto ambience = FindObject("CaveAmbience");
if (ambience != engine::ecs::kNull) {
    if (IsAudioPlaying(ambience)) PauseAudio(ambience);
    else if (IsAudioPaused(ambience)) ResumeAudio(ambience);
    else PlayAudio(ambience, false);
}
```

No-code trigger actions

Add Trigger Audio Action to a trigger object, choose the target runtime name, and select separate enter and exit actions: None, Play, Restart, Pause, Resume, or Stop. This is ideal for doors, ambience zones, and simple level interactions.

Animation events

Event name	Result
Audio.Play	Play the animated object's Audio Source.
Audio.Restart:FootstepAudio	Find FootstepAudio and restart its Audio Source.
Audio.Pause:Radio	Pause Radio.
Audio.Resume:Radio	Resume Radio.
Audio.Stop	Stop the animated object's Audio Source.

TARGETING With no suffix, the event targets the animated entity. A colon suffix targets the object with that runtime name.

Part II — Particle Systems

Author, preview, compose, and control effects

Particle workflow at a glance

Area	What it controls
Playback	Enabled, Autoplay, Loop, Prewarm, duration, start delay, simulation speed, local/world space, and backend.
Emission	Rate, maximum particle count, timed bursts, lifetime, and spawn shape.
Spawn modules	Initial velocity, initial rotation, color, size, and authored variations.
Update modules	Forces, color/size over life, rotation, collision, and trails.
Renderer	Billboard or mesh output, texture/flipbook, blend mode, alignment, scale, and bounds.
Particle Effect	Multiple named emitter layers saved together as a reusable .particlefx effect.

Create a particle system

1. Add Particle System from the component Add menu, or open the Particle Editor panel.
2. Set playback behavior and choose Auto backend unless a specific CPU/GPU path is required.
3. Configure emission rate, lifetime, maximum count, burst behavior, and spawn shape.
4. Add Spawn, Update, and Render modules. Search the module list as the stack grows.
5. Preview, pause, scrub, burst, and clear until timing and silhouette are correct.
6. Save as a reusable .particle preset, or compose multiple layers in a .particlefx effect.

AUTO BACKEND Auto chooses the available implementation. GPU simulation requires an OpenGL 4.3-capable path and falls back safely; use CPU while diagnosing behavior or script-count queries.

Playback and Emission Settings

Playback

Setting	Use
Enabled	Allows simulation and rendering.
Autoplay	Starts when runtime playback initializes.
Loop	Repeats the configured duration.
Prewarm	Simulates ahead so a looping effect appears established immediately.
Duration	Length of one system cycle.
Start Delay	Wait before emission begins.
Simulation Speed	Scales particle-system time without changing game time.
Local Space	Particles follow the emitter transform; world space leaves spawned particles behind.
Backend	Auto, CPU, or GPU simulation selection.

Emission and shapes

Control	Meaning
Rate	Particles emitted per second during continuous playback.
Max particles	Capacity limit; keep it proportional to rate × lifetime plus bursts.
Burst count / interval	Discrete groups emitted immediately or on a repeating interval.
Lifetime	How long a particle remains alive.
Point	All particles originate from the emitter position.
Sphere	Particles spawn within or around a radius.
Cone	Particles use a direction and cone angle for directional sprays.

CAPACITY RULE A useful starting capacity is rate × maximum lifetime + largest overlapping bursts, with some headroom. A low maximum silently flattens the effect under load.

Modules, Rendering, and Bounds

Module stack

Modules are organized into Spawn, Update, and Render stages. Force, Initial Velocity, Rotation, Color Over Life, and Size Over Life can be added more than once and named. Force, velocity, and rotation contributions are additive; color and size are multiplicative, including their curves.

Module	Common use
Initial Velocity	Launch direction, spread, and speed at spawn.
Force	Gravity, wind, lift, or directional acceleration over time.
Rotation	Initial orientation and spin.
Color Over Life	Fade, brighten, tint, or shift hue through normalized lifetime.
Size Over Life	Grow, shrink, pulse, or taper particles.
Collision	Bounce or kill particles against the collision/preview ground.
Trails	Create ribbons or trails behind moving particles.

Renderer

- Billboard faces particles toward the camera; Mesh renders Cube, Sphere, Cone, Cylinder, or Model geometry.
- Velocity alignment rotates suitable particles or meshes along movement direction.
- Additive blend suits fire, sparks, and energy; Alpha blend suits smoke, dust, and soft opaque particles.
- Texture and flipbook settings animate sprite sheets over particle lifetime.
- Mesh scale is separate from object scale; tune both deliberately.

Bounds and culling

The renderer uses bounds to decide whether an effect can be skipped outside the camera view. Make bounds large enough to contain the full effect, especially fast particles, world-space trails, and forces that pull particles away from the emitter.

INVISIBLE OFF-SCREEN EFFECT If a system simulates but disappears too early, visualize or enlarge its bounds before changing emission or material settings.

Particle Editor and Multi-Emitter Effects

Preview controls

Control	Use
Play / Pause	Run or freeze simulation without discarding state.
Restart	Reset time and begin from the configured start state.
Stop	Stop emission; runtime behavior may preserve living particles until cleared.
Burst	Emit the configured burst or an explicit test count.
Clear	Remove all live particles.
Scrub	Inspect timing and curves at a specific point.

The preview scene also provides orbit navigation and presentation controls such as grid, ground, background, and bloom. These are preview aids and do not automatically become level settings.

Reusable assets

- .particle stores a reusable single-emitter preset.
- .particlefx stores a layered effect with multiple emitters, names, and composition settings.
- Open, Save, Revert, and unsaved-change protection support iterative authoring.
- The compiled pipeline graph and validation view help expose missing paths, invalid module values, and unsupported combinations before runtime.

Layering example

Layer	Role	Suggested renderer
Flash	Very short bright core	Additive billboard
Sparks	Fast directional fragments	Additive billboard or mesh
Smoke	Slow expanding body	Alpha billboard
Debris	Heavier falling pieces	Mesh + collision

Particle Scripting Quick Start

Particle helpers work with an object that owns a Particle System or a composed Particle Effect. Use the self overload when the script and effect share an object; pass an entity when a controller targets another object.

Complete example — control separate audio and particle objects

```
#include <engine/gameplay/Script.h>
#include <engine/gameplay/ScriptRegistry.h>
#include <memory>

class ExplosionController final : public engine::Script {
public:
    void OnCreate() override {
        audio_ = FindObject("ExplosionAudio");
        particles_ = FindObject("ExplosionFX");
    }

    void Explode() {
        if (audio_ != engine::ecs::kNull) PlayAudio(audio_, true);
        if (particles_ != engine::ecs::kNull) {
            RestartParticles(particles_);
            BurstParticles(particles_, 120);
        }
    }

private:
    engine::ecs::Entity audio_ = engine::ecs::kNull;
    engine::ecs::Entity particles_ = engine::ecs::kNull;
};

void RegisterExplosionController() {
    engine::ScriptRegistry::Instance().Register(
        "ExplosionController",
        [] { return std::make_unique<ExplosionController>(); });
}
```

REGISTRATION Call RegisterExplosionController() during game startup before entering Play mode, then attach ExplosionController by its registered name.

Particle Script API

Method	Purpose / behavior
PlayParticles(restart = false)	Begin or continue emission; true resets before playing.
StopParticles(clear = false)	Stop emission. Pass true to also remove all living particles.
RestartParticles()	Reset the system and start it again.
BurstParticles(count = 0)	Emit a burst. A count of 0 or less uses the configured burst count.
ClearParticles()	Immediately remove all living particles.
SetParticlesEnabled(enabled)	Enable or disable the component.
SetParticleRate(particlesPerSecond)	Change continuous emission rate at runtime.
SetParticleSpeed(simulationSpeed)	Scale simulation speed for slow motion or acceleration.
AreParticlesPlaying()	Query whether the system is currently playing.
ParticleCount()	Read the component's current emitter count; validate count-sensitive logic when forcing GPU simulation.

Stop versus clear

Call	Visible result
StopParticles(false)	New emission stops; existing particles finish their lifetimes.
StopParticles(true)	Emission stops and existing particles disappear immediately.
ClearParticles()	Existing particles disappear; playback state is otherwise controlled separately.
RestartParticles()	Old state is reset and the effect begins again.

Runtime tuning

```
void SetCombatIntensity(float intensity) {
    intensity = std::clamp(intensity, 0.0f, 1.0f);
    SetParticleRate(12.0f + intensity * 68.0f);
    SetParticleSpeed(0.8f + intensity * 0.7f);
    SetAudioVolume(0.4f + intensity * 0.6f);
}
```

The example uses std::clamp, so include <algorithm>. Apply settings when intensity changes, not continuously unless the value is genuinely animated.

Particle Events Without Custom Code

Trigger actions

Use the particle trigger-action component when an effect only needs to react to entering or leaving a volume. Available actions are None, Play, Restart, Stop, Burst, and Clear. Set the target to the effect object's runtime name.

Animation events

Event name	Result
Particle.Play	Play the animated object's particle component.
Particle.Restart:MuzzleFlash	Restart the named MuzzleFlash object.
Particle.Stop	Stop the animated object's particle component.
Particle.Burst:FootDust	Burst the named FootDust object.
Particle.Clear:WeaponTrail	Clear the named WeaponTrail object.

Choose the simplest controller

Need	Best option
Enter/exit a level volume	Trigger action component
Synchronize with a specific animation frame	Animation event
React to game state, score, health, or AI	engine::Script helper
Coordinate several objects and timed stages	Controller script

EVENT DESIGN Name audio and particle objects by role—MuzzleFlash, FootDust, DoorAudio—so animation-event suffixes remain readable and stable.

Combined Gameplay Patterns

Timed effect sequence

Charge-up pattern — continuously tune, then release once

```
class ChargedEffectScript final : public engine::Script {
public:
    void OnCreate() override {
        SetAudioLooping(true);
        SetParticleRate(8.0f);
        PlayAudio(true);
        PlayParticles(true);
    }

    void OnUpdate(float dt) override {
        elapsed_ += dt;
        const float t = std::min(elapsed_ / 2.0f, 1.0f);
        SetAudioPitch(0.8f + 0.6f * t);
        SetParticleRate(8.0f + 72.0f * t);

        if (!released_ && t >= 1.0f) {
            released_ = true;
            SetAudioLooping(false);
            PlayAudio(true);
            BurstParticles(160);
            StopParticles(false);
        }
    }

private:
    float elapsed_ = 0.0f;
    bool released_ = false;
};
```

This class requires <algorithm> for std::min. Register it using the same ScriptRegistry pattern shown earlier.

Lifecycle guidance

- OnCreate: cache target entities and set initial routing, spatial, attenuation, rate, and speed.
- OnUpdate: respond to one-time events and update values that genuinely change over time.
- OnFixedUpdate: reserve for physics-timed decisions; audio and visual presentation usually belongs in OnUpdate.
- OnDestroy: stop persistent loops or clear effects if they must not survive object teardown.

Troubleshooting

Audio is silent

Check	What to verify
Component	Audio Source exists, is enabled, and has a valid clip.
Transport	Autoplay is appropriate, or PlayAudio / trigger / event actually fired.
Levels	Source volume, bus volume, Master volume, and mute states.
Routing	The source is assigned to the intended AudioBus.
Spatial	Listener exists; source is within useful min/max distance; rolloff is reasonable.
Runtime	Play mode has an audio device/runtime. Helper methods may return false when unavailable.
Replay	Use PlayAudio(true) to retrigger a one-shot that is already playing.

Particles are invisible

Check	What to verify
Playback	Enabled, Autoplay/Play, duration, start delay, and simulation speed.
Emission	Rate or burst is above zero; lifetime and max-particle capacity are sufficient.
Renderer	Valid texture/model, visible color/alpha/size, correct blend mode and mesh scale.
Transform	Local/world space and emitter position are where expected.
Bounds	Culling bounds contain the complete motion and trails.
Backend	Use Auto or CPU to isolate GPU capability/fallback issues.

Script call returns false or does nothing

- Confirm the target is not engine::ecs::kNull and its runtime name matches FindObject exactly.
- Confirm the target owns the required Audio Source, Particle System, or Particle Effect component.
- Confirm the script was registered before Play mode and attached using the same registered name.
- Check the Console for missing asset, decoder, backend, or runtime-service messages.
- Log or branch on bool return values during setup; do not silently assume the component exists.

Release Checklist

Use before committing a scene, prefab, or gameplay feature

Audio

- Clips load and preview without decoder errors.
- Each source uses the correct bus, spatial mode, loop, and autoplay setting.
- Spatial attenuation has been tested from near, mid, and far listener positions.
- Bus and Master headroom remain clean when several sounds overlap.
- Snapshots return to Default on every exit path.
- One-shots restart intentionally; loops have a defined stop or cleanup path.

Particles

- The effect previews correctly from multiple camera angles and distances.
- Max particle count covers the intended rate, lifetime, and overlapping bursts.
- Bounds contain fast particles, world-space motion, collision, and trails.
- CPU and Auto/GPU behavior have been checked for the target hardware path.
- Texture, model, and preset paths remain inside project content.
- Stop, clear, restart, looping, and object-destruction behavior are intentional.

Scripts and events

- Every script is registered before Play mode and attached by its registered name.
- FindObject targets are checked against engine::ecs::kNull.
- Audio and particle commands fire from state changes, not unguarded every frame.
- Animation-event names and target suffixes match scene runtime names.
- No-code trigger actions are used where custom code would add no value.

DONE A feature is ready when editor preview, Play-mode behavior, script/event control, cleanup, and failure handling all match the intended gameplay—not only when the asset looks or sounds correct in isolation.